Plan de Estudio v2: De Senior Backend a AI Software Engineer

Perfil: Backend Senior con Node.js/TypeScript y Ruby on Rails
Disponibilidad: 20 h/semana (4 h × 5 días)
Arranque: miércoles 14 de octubre de 2026 (Semana 0) · **Semana 1: lunes 19 de octubre de 2026**
Cierre estimado: 30 de abril de 2027 · **14 de mayo de 2027** usando el buffer restante
Versión: 2.0 — revisada contra el estado del ecosistema en octubre de 2026

Qué cambió desde la v1

La v1 se escribió a finales de agosto de 2026 sobre conocimiento base de mayo. Seis cambios la afectan de verdad:

#	Cambio	Impacto
1	**Ventanas de 1M tokens como estándar** y precios notablemente más bajos en toda la gama	Reescribe la economía del plan. El cálculo "¿recupero o meto todo en contexto?" cambia de respuesta
2	**Context engineering** se consolidó como disciplina propia	Deja de ser un detalle de prompting y pasa a ser eje transversal, con su propio bloque
3	**La recuperación se movió de *push* a *pull*** (arquitectura de contexto / recuperación agéntica)	La Fase 3 se rebalancea: el RAG clásico sigue, pero deja de ser el único patrón
4	**LangGraph.js sigue por detrás de la versión Python**	Cambia la recomendación de framework: Vercel AI SDK + capa durable, o Mastra
5	**El ecosistema Ruby maduró de verdad** (RubyLLM)	Corrijo mi consejo de la v1: ya no necesitas salirte de Rails para la mayor parte del trabajo
6	**Nuevo OWASP Top 10 para aplicaciones agénticas (2026)** y protocolos de agentes estabilizados (A2A 1.0)	Actualiza el temario de seguridad y añade la capa de protocolos

El esqueleto —8 fases, 26 semanas, proyecto + entrevista + checklist por módulo— se mantiene. Lo que cambia es el contenido de varias fases y el orden de prioridades dentro de ellas.

---

1. Calendario

Semana 0 — hoy, mié 14 a vie 16 de octubre (12 h)

No es estudio, es montar el puesto de trabajo. Si esto no existe, la Semana 1 arranca torcida.

- Repo `ai-engineering-path` creado, con `PLAN.md`, `progress/` y plantilla de log diario.
- Cuentas y API keys de dos proveedores, con límite de gasto configurado.
- Postgres local con pgvector corriendo, y un "hola mundo" que inserta y consulta un vector.
- Proyecto en Claude creado, con `PLAN.md` adjunto como contexto permanente.
- Los 20 bloques de 4 h de las próximas 4 semanas, puestos en el calendario como eventos reales.

Fases

Fase	Semanas	Fechas	Módulo	Horas
0	1–2	19 oct – 30 oct 2026	Nivelación y economía de la inferencia	40
1	3–5	2 nov – 20 nov 2026	LLM API Engineering + Context Engineering	60
2	6–8	23 nov – 11 dic 2026	Embeddings y Vector DBs	60
—	—	**21 dic – 1 ene**	**Pausa navideña (2 semanas de buffer)**	—
3	9–12	14 dic 2026 – 22 ene 2027	Recuperación: RAG y arquitectura de contexto	80
4	13–16	25 ene – 19 feb 2027	Agentes, protocolos y orquestación	80
5	17–19	22 feb – 12 mar 2027	Routing de coste, fine-tuning y modelos abiertos	60
6	20–23	15 mar – 9 abr 2027	LLMOps, seguridad y gobierno	80
7	24–26	12 abr – 30 abr 2027	Capstone y preparación de entrevistas	60

Sobre la pausa navideña: la Fase 3 es la más importante del plan y cae justo encima de las fiestas. En vez de fingir que vas a trabajar el 25 de diciembre, el calendario ya incluye dos semanas de pausa (21 dic – 1 ene) tomadas del buffer. Retomas el 4 de enero a mitad de la Fase 3. Te quedan **2 semanas de comodín** para imprevistos reales; si no las usas, terminas el 30 de abril.

Días festivos que caen en bloque de estudio: 16 nov (S5), 1 feb (S16), 15 mar (S22), 26 mar Viernes Santo (S23). Son 4 días, 16 horas. Absórbelos moviendo el bloque a otro día de esa semana, no acumulando deuda.

Distribución semanal

Bloque	Horas	Qué haces
Implementación guiada	8 h	Código siguiendo docs y repos de referencia
Proyecto del módulo	6 h	El entregable, sin tutorial
Lectura técnica dirigida	3 h	Docs, papers aplicados, post-mortems
Evaluación y testing	2 h	Evals, tests, medición
Escritura técnica	1 h	ADRs, log, borradores

Forma del bloque diario de 4 h

- **0:00–0:15** — relees el objetivo de la semana y el log de ayer
- **0:15–2:15** — bloque profundo 1 (implementación, sin notificaciones)
- **2:15–2:30** — descanso real, lejos de la pantalla
- **2:30–3:45** — bloque profundo 2
- **3:45–4:00** — escribes el log del día

Esos últimos 15 minutos sostienen todo el sistema de seguimiento. Sin log no hay revisión semanal posible.

FASE 0 — Nivelación y economía de la inferencia (S1–2 · 19–30 oct · 40 h)

> **Cambio v2:** esta fase se reescribió casi entera. La v1 enseñaba a razonar sobre ventanas de 200k tokens y sobre un coste por token que ya no es el vigente. Con 1M de contexto como estándar y precios bastante más bajos, varias decisiones de arquitectura que antes eran obvias ahora hay que calcularlas.

Semana	Foco	Horas
1	Modelo mental del LLM + tokenización	20
2	Economía de la inferencia y primer benchmark propio	20

Temas

- **Modelo mental del transformer:** atención, por qué el coste crece con el contexto, y **context rot**: el recall se degrada conforme crece la entrada, aunque quepa. Tener 1M de ventana no significa que convenga llenarla.
- **Los cuatro modos de fallo del contexto** (vocabulario que vas a usar el resto del plan): *poisoning* (un error se repite y se propaga), *distraction* (el modelo se obsesiona con contexto largo), *confusion* (información irrelevante), *clash* (información contradictoria).
- **Tokenización:** BPE, el sobrecoste del español frente al inglés, impacto en factura y en límites efectivos.

- **Parámetros de inferencia:** temperature, top-p, stop sequences, y por qué la reproducibilidad exacta no existe.
- **Economía de la inferencia (rehacer los números tú mismo):** precio de entrada vs. salida, descuentos por batch, prompt caching. Con los precios actuales, el salto entre el modelo más barato y el más capaz es de aproximadamente dos órdenes de magnitud, y eso es exactamente lo que hace rentable el routing que verás en la Fase 5.
- **La pregunta nueva:** ¿recuperar o meter el documento entero en contexto? Construye tu propio árbol de decisión con costes reales, no con intuición. Esta pregunta no existía en la v1 y hoy es la primera que te van a hacer en una entrevista.
- **Python de supervivencia:** entornos, notebooks, leer repos. No migras tu stack.

Evaluación · Fase 0

> **PROYECTO INTEGRADOR Banco de pruebas y calculadora de decisión.** Un script (TS o Ruby) que ejecute la misma tarea contra 4 modelos de distintos tiers y produzca: latencia p50/p95, TTFT, tokens in/out, coste por 1.000 ejecuciones y una métrica de calidad definida por ti. Añade un segundo entregable: una **calculadora de "recuperar vs. contexto largo"** que, dado un tamaño de corpus, una frecuencia de consulta y un precio por token, te diga cuál de las dos estrategias sale más barata y dónde está el punto de cruce. Documéntalo en un ADR de una página.

> **PREGUNTAS DE ENTREVISTA (ARQUITECTURA)**
>
> 1. Tienes 1M de ventana de contexto. Explica por qué montarías RAG igualmente, con argumentos de coste, latencia y calidad.
> 2. Un stakeholder pide "el modelo más potente para todo". Argumenta técnicamente y propón la alternativa.
> 3. Tu producto opera en español. Explica el impacto de la tokenización en tus límites de contexto y en el pricing que ofreces al cliente.
> 4. Define *context rot* y da un caso concreto donde añadir más contexto empeoró un sistema.

> **CHECKLIST DE DOMINIO**
>
> - [ ] Puedo estimar el coste de una feature de IA antes de escribirla.
> - [ ] Tengo una calculadora propia de recuperación vs. contexto largo con números actuales.
> - [ ] Sé explicar TTFT vs. latencia total y por qué importa para UX.
> - [ ] Puedo nombrar los cuatro modos de fallo del contexto y reconocerlos en un sistema real.

Recursos

- *AI Engineering* (Chip Huyen, O'Reilly) — fundamentos y evaluación. Sigue siendo la mejor base, pero **contrasta todos sus números con el pricing vigente**.
- Documentación oficial de modelos y pricing de tus dos proveedores. Es la única fuente fiable de precios.
- Escritos sobre context engineering y *context rot* (ver recursos de la Fase 1).
- Blog de Simon Willison para seguimiento del ecosistema sin ruido comercial.

FASE 1 — LLM API Engineering + Context Engineering (S3–5 · 2–20 nov · 60 h)

> **Cambio v2:** se añade context engineering como bloque de primera clase. La v1 trataba el prompt como la unidad de trabajo; hoy la unidad es **la ventana completa** —instrucciones, definiciones de herramientas, documentos recuperados, memoria, historial, resultados de herramientas— y se recalcula en cada turno.

Semana	Foco	Horas
3	Prompting sistemático y salidas estructuradas	20
4	Streaming, tool calling, resiliencia	20
5	Context engineering: compactación, presupuesto, tool loadout	20

Temas

Capa de API

- Prompting sistemático: system vs. user, few-shot, descomposición, chain-of-thought donde aporta.
- Salidas estructuradas: JSON Schema, validación con Zod (TS) y dry-schema / ActiveModel (Ruby), estrategia de reintento.
- Tool calling: schemas, ejecución paralela, ciclo `tool_use → tool_result`.
- Streaming: SSE con Fastify/Express y `AbortController` en Node; `ActionController::Live` y Turbo Streams en Rails.
- Resiliencia: timeouts, backoff con jitter, circuit breakers, 429, respuestas truncadas.

- Contabilidad de tokens y coste por request, usuario y feature.
- Prompt caching y su efecto real en latencia y precio.
- Prompts como código: versionado, tests de regresión, feature flags.

Context engineering (nuevo)

- El objetivo: el conjunto más pequeño de tokens de alta señal que consigue el resultado. No el más grande que cabe.
- **Las cinco palancas:** selección, compresión, ordenación, aislamiento y persistencia.
- **Compactación:** resumir la conversación al acercarse al límite y reanudar desde el resumen; limpiar resultados de herramientas antiguos es la forma más barata. Riesgo: compactar de más y perder lo crítico.
- **Memoria externa:** escribir notas o progreso a fichero o store fuera de la ventana y releerlas después.
- **Tool loadout:** mantener las definiciones de herramientas fuera de la ventana hasta que hagan falta. El dato que lo justifica: cinco servidores MCP con 58 herramientas pueden consumir alrededor de 55.000 tokens *antes* de que empiece la conversación.
- **Presupuesto de contexto:** medir cuánta evidencia compra cada token. Devolver extractos relevantes, no páginas completas.

Evaluación · Fase 1

> **PROYECTO INTEGRADOR Microservicio de extracción documental con presupuesto de contexto:**
>
> - Endpoint que devuelve JSON validado contra schema estricto.
> - Endpoint de streaming SSE con cancelación limpia.
> - Middleware de contabilidad de tokens/coste persistido en BD.
> - Reintentos, circuit breaker y fallback a modelo secundario.
> - **Un `ContextBuilder` propio**: ensambla la ventana con presupuesto explícito de tokens por sección (instrucciones / herramientas / evidencia / historial), registra la ocupación real en cada llamada y aplica compactación al superar un umbral.
> - Suite de 30+ casos de test de prompt con aserciones reales.

> **PREGUNTAS DE ENTREVISTA (ARQUITECTURA)**
>
> 1. Diseña el manejo de errores de una API que depende de un proveedor con 99.5 % de disponibilidad y p99 de 30 s. ¿Qué SLA ofreces tú encima?
> 2. Un usuario cierra la pestaña a mitad del streaming. ¿Qué pasa en tu servidor, qué se factura, cómo lo gestionas?
> 3. Tu agente arranca consumiendo 55k tokens solo en definiciones de herramientas. Da tres estrategias para reducirlo y el trade-off de cada una.
> 4. El modelo devuelve JSON inválido el 3 % de las veces. Cuatro estrategias para bajarlo a <0.1 % y su coste.
> 5. ¿Cómo versionas y despliegas un cambio de prompt sin regresiones?

> **CHECKLIST DE DOMINIO**
>
> - [ ] Implemento streaming con cancelación en Node **y** en Rails.
> - [ ] Tengo un patrón reutilizable de reintento + validación de salida estructurada.
> - [ ] Mi `ContextBuilder` reporta la ocupación de la ventana por sección en cada llamada.
> - [ ] Mis prompts están versionados y tienen tests que fallan si degradan.
> - [ ] Puedo mostrar un dashboard de coste por feature.

Recursos

- Escritos de ingeniería sobre **context engineering** y gestión de ventana de los principales laboratorios de modelos — es el material más valioso del módulo.
- Taxonomía de modos de fallo del contexto (poisoning, distraction, confusion, clash) de Drew Breunig.
- Cookbooks oficiales de tus proveedores (GitHub).
- **Vercel AI SDK** — la mejor referencia de patrones de streaming tipado en TypeScript.
- **RubyLLM** — streaming, tool calling y structured output en Rails (ver Fase 4 para el detalle).

FASE 2 — Embeddings y Vector Databases (S6–8 · 23 nov – 11 dic · 60 h)

> **Cambio v2:** el núcleo aguanta bien. Se añade lo que la v1 no cubría: los modos de fallo reales de pgvector en producción y los umbrales concretos de escala a los que deja de ser suficiente.

Semana	Foco	Horas
6	Embeddings, similitud, estrategias de chunking	20
7	pgvector en profundidad: índices, tuning y sus trampas	20
8	Búsqueda híbrida, reranking y proyecto	20

Temas

- **Embeddings:** qué representan, dimensionalidad, modelos multilingües, normalización, similitud coseno vs. producto interno, **Matryoshka embeddings** para reducir dimensiones sin reindexar todo.
- **Chunking como decisión de arquitectura:** fixed-size, estructural, recursivo, con solapamiento, *late chunking*, chunking contextualizado.
- **pgvector a fondo** — sigue siendo tu opción por defecto:
  - HNSW vs. IVFFlat; parámetros `m`, `ef_construction`, `ef_search`, `lists`, `probes`.
  - `maintenance_work_mem` **durante la construcción del índice**: con el valor por defecto, construir un índice HNSW sobre una tabla grande se queda sin memoria o tarda muchísimo. Súbelo para el build y devuélvelo después.
  - **Sequential scans silenciosos:** una consulta puede ignorar el índice sin avisar, por desajuste de operador/ops-class, cast implícito de tipos o índice ausente. Disciplina obligatoria: `EXPLAIN ANALYZE` en toda consulta de similitud.
  - **Trampa pre-filter / post-filter:** si la búsqueda vectorial corre antes del filtro, tu `LIMIT` devuelve menos resultados válidos de los que crees. Solución: sobre-recuperar candidatos y filtrar después.
  - **Contención de escritura:** inserciones continuas actualizan el grafo HNSW y suben la p95 de lectura.
  - **Caché frío tras deploy:** pico de p99 en las primeras consultas; mitiga con `pg_prewarm`.
  - `halfvec` (vectores de 16 bits) y reducción de dimensionalidad para recortar memoria y almacenamiento.
- **Umbrales de escala concretos** (memorízalos, son respuesta de entrevista):
  - Hasta ~10M de vectores: pgvector con HNSW va bien.
  - 10M–50M: viable, pero necesitas `halfvec`, presupuesto de memoria cuidado y probablemente **pgvectorscale** (StreamingDiskANN y cuantización binaria; ojo: no está disponible en AWS RDS).
  - Más de 50M: cotiza una base vectorial dedicada, aunque Postgres a menudo siga saliendo más barato.
  - Límites estructurales: Postgres escala vertical, sin sharding horizontal nativo; el índice HNSW debe caber en RAM salvo con pgvectorscale; en la práctica, unas 2.000 dimensiones de techo.
- **Búsqueda híbrida:** full-text de Postgres + vectorial, fusión con **RRF**.
- **Reranking:** cross-encoders y APIs de rerank; por qué recuperar 50 y rerankear a 5 gana a recuperar 5.
- **Operaciones:** reindexado sin downtime, versionado del modelo de embeddings, backfill incremental.

Evaluación · Fase 2

> **PROYECTO INTEGRADOR Motor de búsqueda semántica sobre corpus real:**
>
> - Ingesta idempotente con detección de cambios por hash y backfill incremental.
> - 3 estrategias de chunking comparadas con métricas (Recall@k, MRR, NDCG) sobre 50 queries anotadas por ti.
> - Búsqueda híbrida con RRF + reranking.
> - Benchmark de HNSW con 3 configuraciones de parámetros.
> - **Un test que falla si alguna consulta de similitud cae en sequential scan** (parsea el `EXPLAIN`). Este test es el detalle que separa a un senior.

> **PREGUNTAS DE ENTREVISTA (ARQUITECTURA)**
>
> 1. Tienes 40M de chunks, p95 < 100 ms y filtrado por `tenant_id` y `fecha`. ¿pgvector, pgvectorscale o base dedicada? Justifica con números.
> 2. Tu p95 de búsqueda se duplicó sin cambios de código ni de volumen. Enumera cinco causas en orden de probabilidad.
> 3. Sale un modelo de embeddings mejor. Diseña la migración de 40M de vectores sin downtime.
> 4. ¿Por qué la búsqueda puramente vectorial falla con códigos de producto y nombres propios, y cómo lo resuelves?
> 5. ¿Cómo aíslas datos entre tenants? Compara índice-por-tenant, namespace y filtrado por metadata en coste, seguridad y rendimiento.

CHECKLIST DE DOMINIO

- [ ] Justifico mi chunking con métricas, no con intuición.
- [ ] Sé tunear HNSW y explicar el trade-off que elijo.
- [ ] Reconozco los seis modos de fallo de pgvector en producción y sé diagnosticar cada uno.
- [ ] Conozco de memoria los umbrales de escala y qué hacer en cada tramo.
- [ ] Tengo plan de reindexado sin downtime escrito y probado.

Recursos

- README del repo **pgvector** — entero, incluida la sección de tuning.
- Documentación de **pgvectorscale** (Timescale) — StreamingDiskANN y cuantización binaria.
- Gema `neighbor` (Rails) / integración pgvector en Node.
- Documentación de full-text search de PostgreSQL (`tsvector` , `ts_rank`).
- Documentación de **Qdrant** sobre híbrida y cuantización — buen material conceptual aunque no lo uses.

FASE 3 — Recuperación: RAG y arquitectura de contexto (S9–12 · 14 dic – 22 ene · 80 h)

Cambio v2 — el más importante del plan. La v1 enseñaba RAG clásico: trocear, embeber, recuperar y *empujar* el contexto a la llamada antes de invocar al modelo. El movimiento del mercado es hacia la **arquitectura de contexto**: el agente *tira* de los datos en tiempo de ejecución mediante llamadas a herramientas, y la capa de datos se trata como recurso vivo.

El RAG clásico no está muerto —los titulares que lo anuncian se equivocan cada año— pero dejó de ser el patrón único. Lo que se te va a pedir en una entrevista senior es **elegir entre tres estrategias y defender la elección**: recuperación pre-cargada, recuperación agéntica por herramientas, o contexto largo directo.

Semana	Foco	Horas
9	Pipeline de ingesta industrial	20
—	**Pausa navideña (21 dic – 1 ene)**	—
10	Query understanding y recuperación avanzada	20
11	Recuperación agéntica y capa semántica	20
12	Evaluación sistemática + proyecto	20

Temas

Ingesta (sin cambios respecto a v1)

- Parsing de PDF (texto, escaneado, tablas), HTML, DOCX; extracción de tablas; OCR; normalización a Markdown.
- Pipeline como job en Sidekiq/ActiveJob/Solid Queue o BullMQ, con reintentos, DLQ y observabilidad.
- Enriquecimiento: metadata por LLM, resúmenes por documento, preguntas hipotéticas por chunk.

Query understanding y recuperación avanzada

- Reescritura de consulta, multi-query, HyDE, descomposición, routing entre fuentes.
- Parent-document retrieval, sentence-window, recuperación jerárquica, contextual retrieval.
- Ensamblado de contexto: ordenación, deduplicación, presupuesto de tokens, compresión.
- Citas y verificabilidad: atribución forzada, mapeo al offset original.
- Anti-alucinación: groundedness checks, "no lo sé" como comportamiento de primera clase.

Arquitectura de contexto (nuevo)

- **Push vs. pull:** el contraste central. En RAG clásico decides por adelantado qué necesita el modelo; en recuperación agéntica expones herramientas de búsqueda y dejas que el modelo decida qué traer y cuándo.
- **Diseño de herramientas de recuperación** como si fueran API: `search_docs` , `get_document_section` , `list_recent` . Devuelven extractos relevantes, no páginas enteras.
- **El árbol de decisión de tres ramas**, con criterios de coste, latencia, frescura y verificabilidad — y sabiendo que un sistema real suele combinar las tres.
- **Capa semántica:** definir entidades, métricas y relaciones de negocio una sola vez, de forma legible por máquina, para que el agente no calcule números con confianza y sin razón. Es lo que diferencia un demo de un sistema en el que confía finanzas.
- **CDC y frescura:** sincronización continua desde las bases relacionales; qué dato debe estar fresco al segundo y cuál puede tener

horas.
- **Control de acceso del lado del servidor:** los permisos a nivel de fila se aplican en la recuperación, no en el prompt. Un agente que pide datos nunca es la autoridad sobre qué puede ver.

Evaluación (lo más importante del módulo)

- Métricas de recuperación: context precision, context recall.
- Métricas de generación: faithfulness, answer relevancy.
- Golden set construido a mano, mínimo 100 casos.
- LLM-as-a-judge: cómo calibrarlo contra juicio humano y sus sesgos conocidos.
- Evals en **CI**, bloqueando el merge si degrada.
- Caché semántico: cuándo ayuda, cuándo es peligroso, umbrales, invalidación.

Evaluación · Fase 3

PROYECTO INTEGRADOR Servicio de recuperación multi-tenant con las tres estrategias:

- API con autenticación, cuotas por tenant y aislamiento estricto aplicado en la capa de datos.
- Ingesta asíncrona con estados observables y reprocesado.
- **Las tres rutas implementadas y seleccionables por configuración**: RAG pre-cargado, recuperación agéntica por herramientas, contexto largo directo.
- **Un informe comparativo de las tres** sobre el mismo golden set: calidad, latencia p50/p95, coste por consulta. Este informe es, probablemente, la pieza más valiosa de todo tu portfolio.
- Respuestas con citas trazables al documento y posición original.
- Golden set de 100 preguntas con evals en cada PR, publicando reporte comparativo.
- Caché semántico con métricas de hit rate.

PREGUNTAS DE ENTREVISTA (ARQUITECTURA)

1. Dado un corpus de 200 documentos de 10 páginas y 50 consultas al día: ¿RAG, herramientas o contexto largo? Ahora súbelo a 2 millones de documentos y 50.000 consultas al día. ¿Qué cambia y por qué?
2. Diseña un sistema de recuperación para 500 clientes enterprise con permisos a nivel de carpeta. ¿Dónde aplicas la autorización y por qué exactamente ahí?
3. Los usuarios reportan que el sistema "inventa". ¿Cómo distingues un fallo de recuperación de uno de generación, con datos?
4. Tu agente consulta métricas de negocio y da una cifra de ingresos distinta a la del dashboard. ¿Qué falló arquitectónicamente y cómo lo arreglas de raíz?
5. Un cliente sube 500.000 documentos de golpe. Diseña la ingesta para que no degrade a los demás tenants.
6. Justifica ante un CTO por qué necesitas dos semanas de evaluación antes de "solo mejorar el prompt".

CHECKLIST DE DOMINIO

- [ ] Tengo golden set propio y evals corriendo en CI.
- [ ] Implementé y medí las tres estrategias de recuperación sobre el mismo benchmark.
- [ ] Puedo defender con números cuándo usar cada una.
- [ ] Diagnostico con datos si un fallo es de recuperación o de generación.
- [ ] Mi pipeline de ingesta es idempotente, reanudable y observable.
- [ ] La autorización vive en la capa de datos, no en el prompt.

Recursos

- **RAGAS** — marco de referencia para métricas de RAG.
- Escritos sobre **contextual retrieval** y recuperación agéntica de los laboratorios de modelos.
- Documentación de **LlamaIndex** sobre patrones de recuperación — la mejor conceptualización disponible, aunque trabajes en TS.
- Material sobre **capa semántica** y definiciones de métricas legibles por máquina (el espacio OSI / Open Semantic Interchange es el estándar emergente aquí).
- *AI Engineering* (Chip Huyen), capítulos de RAG y evaluación.

FASE 4 — Agentes, protocolos y orquestación (S13–16 · 25 ene – 19 feb · 80 h)

> **Cambios v2 — dos, ambos relevantes. (a) Framework:** la v1 recomendaba LangGraph.js como opción principal. El port de TypeScript sigue por detrás de la versión Python en features y documentación. Para tu perfil la apuesta correcta es **Vercel AI SDK + una capa de ejecución durable** (Inngest, Trigger o equivalente), o **Mastra** si prefieres un stack integrado con workflows, memoria y evals de serie, aceptando más lock-in. **(b) Ruby:** me equivoqué en la v1. Dije que el ecosistema Ruby era demasiado delgado y que debías salirte a Node. **RubyLLM** cambió eso: una gema con interfaz única sobre múltiples proveedores, streaming, tool calling, structured output, embeddings, integración con ActiveRecord vía `acts_as_chat`, pgvector con scopes de ActiveRecord y un DSL de agentes. Hay soporte MCP, instrumentación OpenTelemetry y tooling de evaluación en el ecosistema. Rails ya te da jobs, auth, caché, streaming y deploy; meter un servicio aparte añade coste operativo que quizá no necesitas. El matiz honesto: **el mercado laboral conoce más LangChain/TypeScript que RubyLLM.** Para aprender y para tu empresa, Rails es defendible. Para tu portfolio de búsqueda de empleo, construye al menos un proyecto agéntico en TypeScript.

Semana	Foco	Horas
13	Anatomía del agente: loop, herramientas, memoria	20
14	Frameworks y protocolos (MCP, A2A)	20
15	Durabilidad, HITL y control de presupuesto	20
16	Seguridad agéntica + proyecto	20

Temas

- **Anatomía del agente:** el loop, ReAct, y la diferencia real entre un *workflow* determinista con LLM y un agente autónomo. Advertencia que se repite en todas las fuentes serias: muchos sistemas "multi-agente" son un agente con complejidad añadida.
- **Patrones de workflow:** prompt chaining, routing, paralelización, orchestrator-workers, evaluator-optimizer.
- **Diseño de herramientas:** nombres y descripciones son la interfaz de API del LLM; idempotencia obligatoria; validación de argumentos; separación lectura/escritura; confirmación explícita para lo destructivo. **Cada herramienta es una frontera de seguridad.**
- **Sub-agentes:** cada uno explora en su ventana limpia y devuelve un resumen corto. Cuidado: los sub-agentes que no ven las decisiones de los otros producen trabajo inconsistente, así que encajan mejor en investigación de lectura que en escritura paralela.
- **Memoria:** corto plazo, compactación, memoria externa en fichero o store, memoria episódica vs. semántica.
- **Protocolos (bloque nuevo):**
  - **MCP** es la capa de herramientas asentada, con soporte amplio de proveedores y frameworks y miles de servidores. Construye el tuyo en TypeScript. Diseña los servidores como APIs: herramientas pocas, pequeñas y bien descritas.
  - **A2A (Agent-to-Agent):** descubrimiento y delegación entre agentes mediante Agent Cards firmadas. Nació en Google, donado a la Linux Foundation, versión 1.0 desde abril de 2026. Apréndelo, pero úsalo solo cuando de verdad necesites varios agentes.
  - **WebMCP:** expone capacidades de sitios web a los agentes, vía W3C, en preview temprana en Chrome Canary desde febrero de 2026. Es la capa más joven: conócela, no construyas sobre ella todavía.
  - **Capa semántica (OSI):** la pieza que impide que los agentes calculen números equivocados con seguridad.
- **Frameworks:** Vercel AI SDK (streaming de referencia, tool calls tipadas con Zod, cambio de proveedor cambiando un string; no cubre ejecución durable por sí solo), Mastra ("Rails para agentes", integrado, más joven y con APIs que han cambiado más), OpenAI Agents SDK (solo si te atas a esa plataforma), LangGraph (máquina de estados explícita con checkpointing, el más maduro para flujos ramificados complejos, pero más fuerte en Python). En Ruby: RubyLLM con su DSL de agentes.
- **Durabilidad:** un agente que corre 8 minutos no cabe en un request HTTP. Checkpointing, reanudación tras fallo, Sidekiq/Solid Queue o BullMQ, Inngest/Trigger/Temporal.
- **Human-in-the-loop:** pausar, persistir estado, esperar aprobación, reanudar.
- **Control de recursos:** presupuesto de tokens, pasos y coste por ejecución; detección de bucles; kill switch.
- **Seguridad agéntica:** prompt injection directa e indirecta; la *lethal trifecta* (datos privados + contenido no confiable + capacidad de exfiltración); sandboxing; mínimo privilegio y credenciales de vida corta **por agente**; aprobación humana para lo irreversible; auditoría contra identidades reales. Trata toda salida de herramienta y todo contenido recuperado como entrada no confiable.
- **Gobierno como arquitectura:** los agentes necesitan el mismo RBAC, acceso con scope y rastro de auditoría que las personas. La elección de protocolo importa menos que si identidad, permisos y auditoría sobreviven en todas las rutas de acceso.

Evaluación · Fase 4

PROYECTO INTEGRADOR Agente operativo durable (triaje de incidencias, soporte técnico, o lo que conozcas bien):

- Mínimo 5 herramientas reales, al menos una de escritura con confirmación humana.
- Ejecución durable con checkpointing: si matas el proceso a mitad, se reanuda desde el último estado.
- Presupuesto por ejecución (tokens, pasos, coste) con corte duro.
- **Tool loadout dinámico:** las definiciones de herramientas no se cargan todas de entrada. Mide los tokens ahorrados.
- Trazas completas: qué pensó, qué llamó, qué recibió, cuánto costó.
- Un **servidor MCP** que exponga al menos dos de esas herramientas.
- Suite con 3+ escenarios de prompt injection indirecta que el agente debe resistir.
- **Construido en TypeScript** (para el portfolio). Opcional y recomendable: una segunda versión reducida en Rails con RubyLLM, y un ADR comparando ambas experiencias.

PREGUNTAS DE ENTREVISTA (ARQUITECTURA)

1. ¿Cuándo NO usarías un agente? Tres casos reales donde el workflow determinista gana, y el criterio que los distingue.
2. Tu agente llega a 40 pasos y llama a APIs externas. ¿HTTP, cola o workflow engine? ¿Cómo garantizas exactly-once en las escrituras?
3. Un agente lee un email de un atacante con instrucciones embebidas. Diseña las tres capas de defensa que lo impiden.
4. ¿Cuándo justifica A2A su complejidad frente a un agente con más herramientas?
5. ¿Cómo depuras un agente que falla el 5 % de las veces de forma no determinista? ¿Qué instrumentación necesitas desde el día uno?
6. Tienes un monolito Rails. Defiende construir la IA dentro vs. en un servicio aparte. Argumenta las dos posturas y luego elige.

CHECKLIST DE DOMINIO

- [ ] Distingo con criterio propio cuándo un workflow supera a un agente.
- [ ] Mi agente sobrevive al reinicio del proceso a mitad de ejecución.
- [ ] Tengo defensas explícitas y probadas contra prompt injection indirecta.
- [ ] Construí y desplegué un servidor MCP.
- [ ] Puedo explicar el stack de protocolos (MCP, A2A, WebMCP, capa semántica) y qué resuelve cada capa.
- [ ] Reconstruyo cualquier ejecución fallida desde sus trazas.

Recursos

- **Building Effective Agents** (ingeniería de Anthropic) — la mejor taxonomía disponible, sigue vigente.
- Especificación de **MCP** (`modelcontextprotocol.io`) y SDK de TypeScript.
- Especificación de **A2A** en la Linux Foundation.
- Documentación de **Vercel AI SDK** y de **Mastra**; **LangGraph** para el modelo mental de máquina de estados.
- **RubyLLM** — documentación y código fuente.
- **OWASP Top 10 para aplicaciones agénticas (2026)** — ver Fase 6.
- Escritos de Simon Willison sobre prompt injection y la *lethal trifecta*.

FASE 5 — Routing de coste, fine-tuning y modelos abiertos (S17–19 · 22 feb – 12 mar · 60 h)

Cambio v2: se invierte el énfasis. La v1 ponía el fine-tuning en el centro y el routing al final. Con la gama de precios actual — donde el modelo más barato cuesta una fracción minúscula del más capaz— **el routing bien hecho suele dar más ahorro y con mucho menos riesgo operativo que un fine-tuning**. El fine-tuning sigue, pero como herramienta de segunda llamada.

Semana	Foco	Horas
17	Routing, cascadas y optimización de coste	20
18	Destilación y fine-tuning con LoRA/QLoRA	20
19	Serving de modelos abiertos y decisión make-vs-buy	20

Temas

- **La jerarquía de decisión, en orden:** mejor prompt → mejor context engineering → few-shot → recuperación → herramientas →

routing a modelo más pequeño → fine-tuning → modelo más grande. El fine-tuning no arregla falta de conocimiento; arregla formato, estilo, tono y tareas estrechas repetitivas.

- **Routing y cascadas:** clasificador barato que decide destino; umbrales; escalado automático al modelo grande cuando la confianza es baja; medición del ahorro real y de la degradación aceptable.
- **Otras palancas de coste:** prompt caching agresivo, batch APIs para lo asíncrono, compresión de contexto, recorte de evidencia. Mide **coste por unidad de valor de negocio** (por ticket resuelto, por documento procesado), nunca coste total.
- **Curación de datasets:** calidad sobre cantidad, deduplicación, splits, contaminación entre train y eval, generación sintética y sus riesgos.
- **Técnicas:** SFT vs. LoRA vs. QLoRA; rank, alpha, learning rate, épocas; overfitting y *catastrophic forgetting*.
- **Destilación:** el modelo grande genera el dataset del pequeño. Sigue siendo el caso con mejor ROI para un backend engineer.
- **Evaluación del modelo afinado:** contra el modelo base **y** contra el prompting bien hecho. Si no gana en calidad *y* en coste/latencia, no se despliega.
- **Serving de modelos abiertos:** vLLM (continuous batching, PagedAttention), Ollama en local, cuantización, dimensionamiento de GPU, coste real de GPU 24/7 vs. API por token.
- **Plataformas gestionadas:** criterios de elección por latencia, compliance, residencia de datos y disponibilidad.

Evaluación · Fase 5

PROYECTO INTEGRADOR Reducción de coste medida, en dos movimientos. Toma una tarea real de fases anteriores:

1. **Primero el routing:** implementa una cascada modelo-pequeño → modelo-grande con umbral calibrado. Mide ahorro y degradación sobre tu golden set. Documenta el punto donde la calidad deja de ser aceptable.
2. **Después el fine-tuning:** genera 1.000+ ejemplos curados con el modelo grande, afina un modelo pequeño abierto con LoRA/QLoRA, despliégalo tras una API compatible.
3. **Informe final comparando las tres opciones** —modelo grande, cascada, modelo afinado— en calidad, latencia p50/p95, coste por 1.000 requests y esfuerzo operativo, con el punto de equilibrio de volumen para cada una.

La conclusión honesta puede ser que el fine-tuning no valía la pena. **Si es así, escríbelo.** Ese informe vale más en una entrevista que uno que concluye lo que esperabas.

PREGUNTAS DE ENTREVISTA (ARQUITECTURA)

1. Tu CEO leyó que hay que "entrenar un modelo propio con nuestros datos". Explícale en 3 minutos por qué probablemente quiere otra cosa, y en qué escenario concreto sí tendría razón.
2. Procesas 10M de requests/mes. Modela la decisión entre API por token y GPUs propias: variables, punto de equilibrio, riesgos operativos.
3. Diseña un sistema de routing. ¿Cómo calibras el umbral y cómo evitas degradar la experiencia?
4. Tu modelo afinado va mejor en eval y peor en producción. Cinco causas en orden de probabilidad.
5. Tienes que recortar el gasto de IA un 60 % sin perder calidad perceptible. Da tu plan en orden de ejecución y el ahorro estimado de cada paso.

CHECKLIST DE DOMINIO

- [ ] Tengo un árbol de decisión propio y defendible sobre cuándo afinar.
- [ ] Implementé routing con ahorro medido y degradación cuantificada.
- [ ] Ejecuté un fine-tuning completo y lo evalué con honestidad contra el baseline.
- [ ] Sé desplegar y dimensionar un modelo abierto con vLLM.
- [ ] Puedo responder "cómo recorto el coste un 60 %" con un plan ordenado.

Recursos

- Documentación de **Unsloth** y de **Hugging Face TRL/PEFT** para LoRA/QLoRA.
- **Hugging Face smol-course** — práctico y ligero.
- Documentación de **vLLM**.
- *AI Engineering* (Chip Huyen), capítulos de fine-tuning y optimización de inferencia.
- Pricing oficial vigente de tus proveedores — la fuente de todos tus cálculos.

FASE 6 — LLMOps, seguridad y gobierno (S20–23 · 15 mar – 9 abr · 80 h)

> **Cambio v2:** la seguridad se actualiza al marco nuevo. OWASP publicó un **Top 10 específico para aplicaciones agénticas (2026)**, revisado por más de 100 expertos, además de la revisión 2026 del Top 10 para aplicaciones LLM. Se añade gobierno de datos como tema de primera clase: es lo que las empresas están descubriendo que les falta.

Semana	Foco	Horas
20	Observabilidad y tracing	20
21	Evaluación continua, A/B y despliegue seguro	20
22	Seguridad, privacidad y gobierno	20
23	Escala, latencia, fiabilidad + proyecto	20

Temas

- **Observabilidad:** tracing distribuido sobre cadenas LLM, convenciones **OpenTelemetry GenAI**, herramientas (Langfuse, LangSmith, Helicone). Qué guardar (prompts, respuestas, latencias, tokens, coste, versión de modelo y de prompt) y qué no (PII).
- **Métricas que reportas al negocio:** coste por conversación, tasa de deflexión, escalado a humano, latencia percibida, tasa de "no lo sé", satisfacción.
- **Evaluación continua:** online vs. offline, muestreo de tráfico real para revisión humana, detección de drift, alertas por caída de calidad.
- **Despliegue seguro:** shadow mode, canary por porcentaje, A/B con significancia estadística, rollback instantáneo de prompt y de modelo. La versión del modelo es una dependencia que cambia bajo tus pies.
- **Fiabilidad:** multi-proveedor con fallback, degradación elegante, bulkheads, presupuesto de error.
- **Latencia:** streaming como herramienta de UX, prefetching, caché multinivel, paralelización, elección de modelo por percentil.
- **Escala:** colas y backpressure, rate limiting por tenant, cuotas del proveedor, prioridad de trabajos, batch para asíncrono.
- **Seguridad:**
  - **OWASP Top 10 para aplicaciones agénticas (2026)** — el documento central de este módulo.
  - **OWASP Top 10 para aplicaciones LLM (revisión 2026)**.
  - Detección y redacción de PII antes de enviar al proveedor.
  - Retención de datos del proveedor, residencia, DPAs, zero data retention.
  - Guardrails de entrada y salida, moderación.
- **Gobierno (ampliado):** identidad y permisos para agentes igual que para personas; credenciales de vida corta con scope por agente; aprobación humana para lo irreversible; auditoría contra identidades reales. La capa semántica versionada y mantenida con el mismo rigor que un pipeline de datos.
- **Gobierno de costes:** cuotas por tenant, alertas de anomalía, atribución por feature y por cliente, **coste de recuperación por consulta** como métrica de primera clase.

Evaluación · Fase 6

> **PROYECTO INTEGRADOR LLM Gateway propio** — el servicio que toda tu organización usaría para hablar con modelos:
>
> - Interfaz unificada sobre 2+ proveedores con fallback automático y circuit breaker por proveedor.
> - Rate limiting y cuotas de coste por tenant/API key, con corte y alerta.
> - Tracing completo con OpenTelemetry exportado a Langfuse o equivalente.
> - Caché de dos niveles (exacto + semántico) con métricas de hit rate y ahorro.
> - Redacción de PII configurable en la entrada.
> - Versión de prompt y de modelo en cada traza, con rollback.
> - Evals continuos sobre muestra de tráfico real.
> - **Una auditoría escrita de uno de tus proyectos anteriores contra el OWASP Top 10 agéntico**, con hallazgos y remediaciones. Es un artefacto de portfolio excelente y casi nadie lo tiene.
> - **Runbook de incidentes:** "el proveedor está caído", "el coste se disparó 10x", "la calidad cayó tras un cambio de versión del modelo".

PREGUNTAS DE ENTREVISTA (ARQUITECTURA)

1. Tu proveedor actualiza el modelo silenciosamente y la calidad cae un 15 %. ¿Cómo te enteras, en cuánto tiempo, y qué haces en los primeros 30 minutos?
2. Diseña la observabilidad de un sistema de recuperación con agentes desde cero. ¿Qué spans, qué atributos, qué cardinalidad aceptas?
3. Un tenant consume el 70 % de tu cuota del proveedor y degrada al resto. Solución técnica y de producto.
4. Tienes que cumplir GDPR con un proveedor fuera de la UE. ¿Qué decisiones de arquitectura te fuerza?
5. Define el SLO de un endpoint de IA generativa. ¿Qué mides si la corrección no es binaria?
6. ¿Cómo das a un agente acceso a los datos de producción sin darle acceso a los datos de producción?

CHECKLIST DE DOMINIO

- [ ] Trazo una petición completa a través de recuperación + agente + herramientas.
- [ ] Tengo alertas que detectan degradación de calidad, no solo errores 500.
- [ ] Mi sistema sobrevive a la caída total del proveedor principal.
- [ ] Auditè un sistema propio contra el OWASP agéntico y documenté las remediaciones.
- [ ] Escribí un runbook y ejecuté un simulacro de incidente.
- [ ] Atribuyo el coste de IA por cliente y por feature.

Recursos

- **OWASP Top 10 for Agentic Applications 2026** (`genai.owasp.org`) — documento completo.
- **OWASP Top 10 for LLM Applications**, revisión 2026.
- Documentación de **Langfuse** (self-hosted, encaja con tu perfil) y convenciones semánticas **GenAI de OpenTelemetry**.
- **NIST AI Risk Management Framework** — el vocabulario de los equipos de compliance.
- Post-mortems públicos de incidentes con LLMs en producción. Búscalos activamente.

FASE 7 — Capstone y preparación de entrevistas (S24–26 · 12 abr – 30 abr · 60 h)

Semana	Foco	Horas
24	Capstone: construcción end-to-end	20
25	Capstone: endurecimiento, evals y documentación	20
26	System design, portfolio y simulacros	20

Actividades

- **Capstone:** producto real, alcance pequeño, profundidad alta. Integra recuperación + agente + gateway + evals + observabilidad. Idealmente sobre un dominio que ya entiendes.
- **System design de sistemas de IA**, en 45 minutos y en pizarra:
  - Asistente de soporte sobre 1M de documentos.
  - Revisión de contratos con extracción estructurada y validación humana.
  - Copiloto interno con acceso a APIs de la empresa y permisos por rol.
  - Plataforma de evaluación de modelos para 20 equipos internos.
  - **Nuevo:** migrar un RAG clásico existente a arquitectura de contexto, sin romper a los clientes actuales.
- **Portfolio:** 3–4 repos con README de calidad profesional (problema, arquitectura, decisiones, trade-offs, métricas, qué harías distinto). Un README excelente pesa más que diez repos abandonados.
- **Escritura:** 2 artículos sobre algo que mediste y que no es obvio. Tus candidatos naturales son el comparativo de las tres estrategias de recuperación (Fase 3) y el informe de reducción de coste (Fase 5). Ambos tienen números propios, que es exactamente lo que no tiene el contenido genérico que satura el tema.
- **Narrativa de entrevista:** historias STAR sobre trade-offs de coste, incidentes y decisiones de "no usar IA aquí". Un "decidimos no usar un agente porque…" vale más que una lista de tecnologías.

Evaluación final

CHECKLIST DE GRADUACIÓN

- [ ] Tengo un sistema en producción, aunque sea con tráfico pequeño y real.
- [ ] Defiendo cada decisión de arquitectura con datos que yo medí.
- [ ] Sé cuánto cuesta mi sistema por unidad de negocio y cómo reducirlo un 40 %.
- [ ] Diseño en pizarra un sistema de IA a escala en 45 minutos.
- [ ] Articulo los límites de la IA y cuándo no aplicarla.
- [ ] Mi portfolio es navegable y comprensible en 10 minutos.

2. Principios transversales

- **Mide siempre.** La diferencia entre un AI engineer y alguien que usa la API es que el primero tiene números propios.
- **Construye el eval antes que la feature.** Contraintuitivo, y lo que más te diferencia.
- **Tu ventaja es el backend.** No compitas en modelado; compite en fiabilidad, coste, latencia, seguridad y operación. Ese terreno sigue poco cubierto.
- **Trata todo contenido recuperado como entrada no confiable.** Vale para documentos, resultados de herramientas y respuestas de otros agentes.
- **Escribe ADRs.** Uno por módulo. Material de entrevista gratis.
- **Verifica contra documentación vigente.** Este plan tiene fecha de caducidad; lo acabamos de comprobar. Cuando algo del plan contradiga a la documentación oficial del día, gana la documentación.

3. Riesgos y mitigación

Riesgo	Señal temprana	Mitigación
Tutorial hell	3 días siguiendo guías sin código propio	Máximo 40 % del tiempo en material guiado
Perseguir frameworks nuevos	Reescribes el proyecto con la herramienta de moda	Congela el stack al inicio de cada fase
Saltarte las evals	"Se ve bien" como criterio de calidad	El checklist de módulo bloquea el avance
Ambición del capstone	El proyecto crece y no lo terminas	Alcance pequeño, profundidad alta
Obsolescencia	El plan contradice la documentación	Revisión del plan en S13 y S20
Nueva pausa larga	Dos semanas sin log	Retomas en el módulo donde estabas, no desde cero. Relees tu propio log antes que nada

4. Punto de control

Semana 13 (25 de enero de 2027), a mitad de camino. Deberías tener dos proyectos públicos sólidos y poder discutir arquitectura de recuperación con solvencia. Si no tienes evals corriendo en CI en ningún proyecto, detente y arréglalo antes de seguir: el resto del plan se apoya en esa capacidad.

Reserva además una hora en **S13 y S20** para revisar este plan contra el estado real del ecosistema. Acabamos de ver que cinco meses bastan para invalidar varias recomendaciones.